



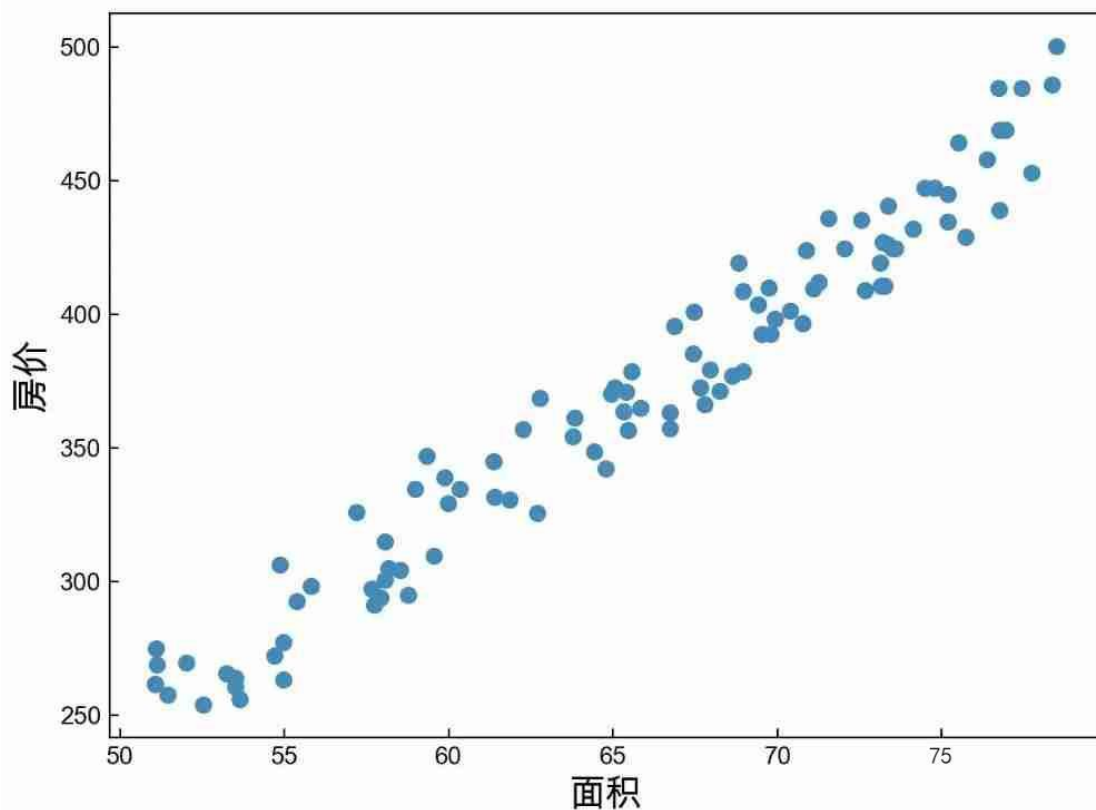
机器学习-线性回归

01 回归问题

02 线性回归模型的建立

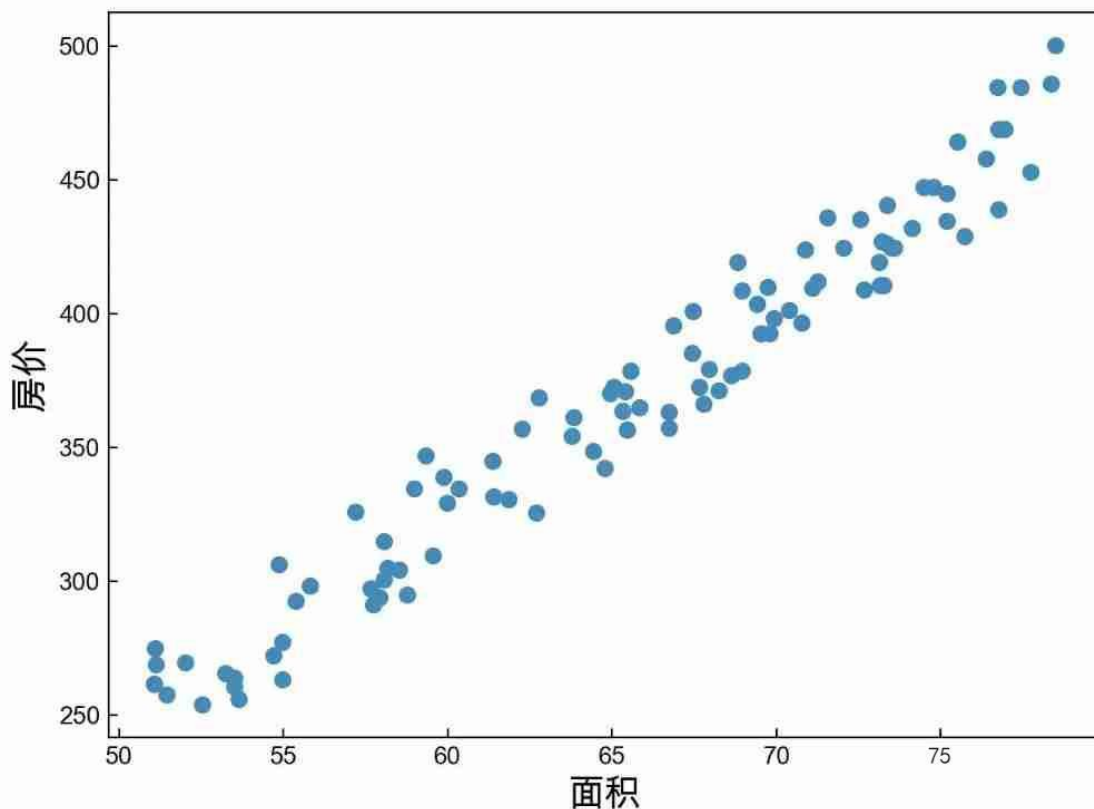
03 线性回归模型的求解

04 线性回归评价指标



某市房价与面积的关系如图所示，其中横坐标为面积，纵坐标为价格。显然随着房屋面积的增加，房价整体上呈线性增长的趋势。

现在随意给出一个房屋的面积，要如何才能预测（或者计算）出其对应的价格呢？



因此可以考虑建立一个如下的线性模型，来拟合已有数据，令其可以推断或预测未知数据：

$$\hat{y} = wx + b$$

其中 w 叫权重参数 (Weight)， b 叫偏置 (Bias) 或者截距 (Intercept)。当求解得到未知参数 w 与 b 之后，也就意味着我们得到了这个预测模型，即给定一个房屋面积 x ，就能够预测出其对应的房价 $\hat{y}$ 。

01 回归问题

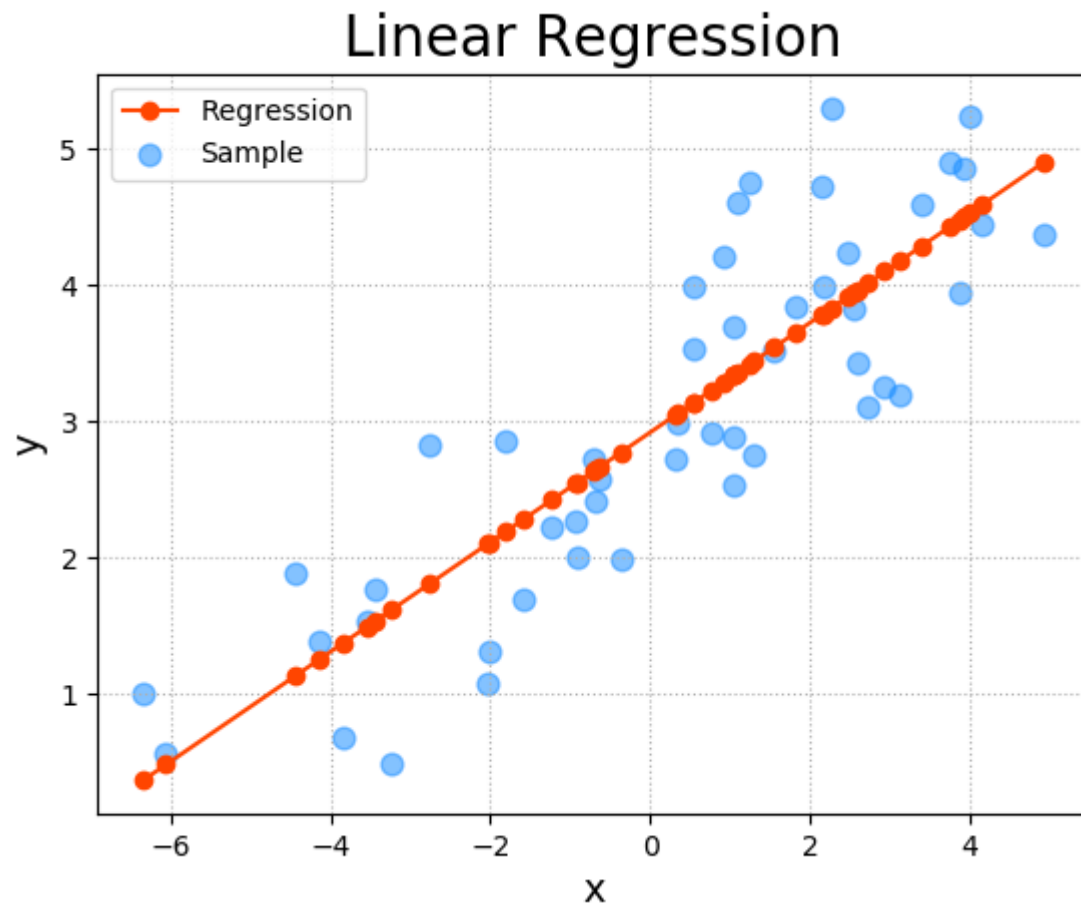
02 线性回归模型的建立

03 线性回归模型的求解

04 线性回归评价指标

线性回归 (Linear Regression)

线性回归是一种统计方法，研究因变量（目标变量）与一个或多个自变量（特征变量）之间的线性关系。通过拟合一条直线，线性回归模型试图求解最优的线性函数，使得预测值与实际观测值之间的误差最小。



线性回归-预测流程

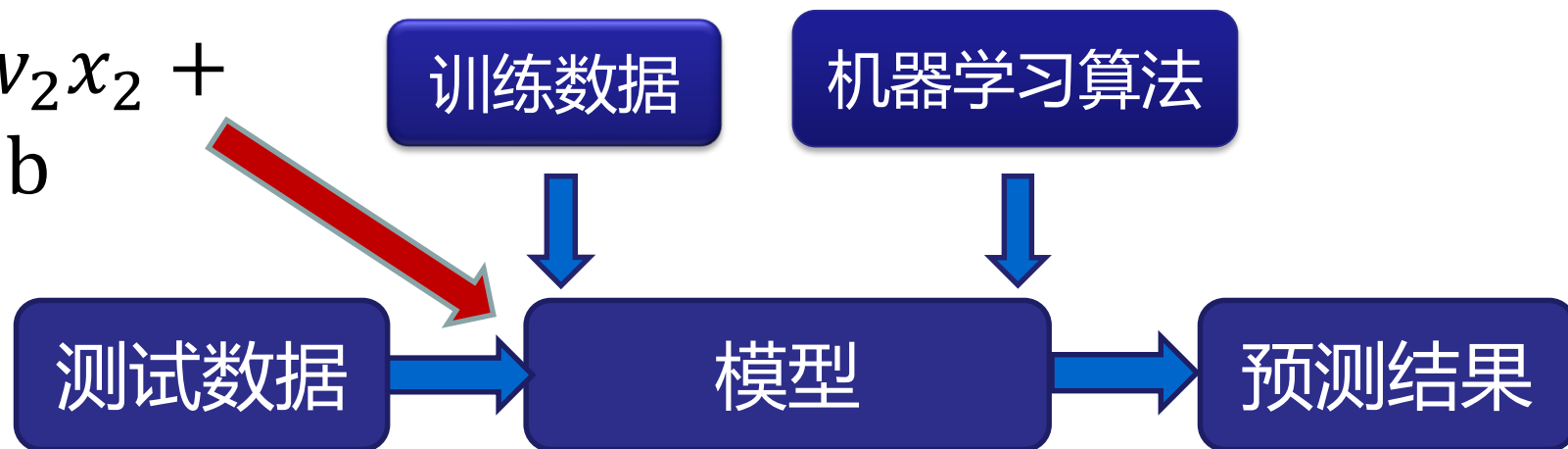


北京邮电大学
Beijing University of Posts and Telecommunications

7

$$h(x) = wx + b$$

$$h(x) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$



注意：若表达式 $h(x) = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n + b$ ，则 b 可以融入到 w_0

01 回归问题

02 线性回归模型的建立

03 线性回归模型的求解

1. 最小二乘法
2. 梯度下降法
3. 多变量特征方程法

04 线性回归评价指标

损失函数(Loss Function): 度量单样本预测的错误程度, 损失函数值越小, 模型就越好。常用的损失函数包括: 0-1损失函数、平方损失函数、绝对损失函数、对数损失函数等。

代价函数(Cost Function): 度量全部训练样本集的平均误差。常用的代价函数包括均方误差、均方根误差、平均绝对误差等。

目标函数(Object Function): 代价函数和正则化函数, 最终要优化的函数。

损失函数采用平方和损失:

$$l(x^{(i)}) = \frac{1}{2} (h(x^{(i)}) - y^{(i)})^2$$

对于 $h(x) = wx + b$, 要确定一组 w 和 b , 使得

$$J(w, b) = \frac{1}{2n} \sum_{i=1}^n (h(x^{(i)}) - y^{(i)})^2$$

(残差平方和) 最小, 这一组 w 和 b 即为该模型所求参数, $J(w, b)$ 即为我们要优化的目标函数。

备注: 在本课程中, 目标函数与代价函数二者等同, 并不涉猎对正则化的讨论及应用。

损失函数的系数1/2是为了便于计算, 使对平方项求导后的常数系数为1, 这样在形式上稍微简单一些。有些教科书把系数设为1/2, 有些设置为1, 这些都不影响结果。

线性回归-最小二乘法模型求解



对于代价函数 $J(w, b) = \frac{1}{2n} \sum_{i=1}^n (h(x^{(i)}) - y^{(i)})^2$

分别对 w 、 b 进行求导：

$$\begin{aligned}\frac{\partial}{\partial w} J(w, b) &= \frac{1}{n} \sum_{i=1}^n x_i (wx_i + b - y_i) \\&= w \left(\frac{1}{n} \sum_{i=1}^n x_i^2 \right) + b \left(\frac{1}{n} \sum_{i=1}^n x_i \right) - \left(\frac{1}{n} \sum_{i=1}^n x_i y_i \right) \\&= w \left(\frac{1}{n} \sum_{i=1}^n x_i^2 \right) + b \bar{x} - \left(\frac{1}{n} \sum_{i=1}^n x_i y_i \right) \\ \frac{\partial}{\partial b} J(w, b) &= \frac{1}{n} \sum_{i=1}^n wx_i + b - y_i = w \bar{x} + b - \bar{y}\end{aligned}$$

线性回归-最小二乘法模型求解

导数为0时，损失函数最小。令

$$\begin{cases} \frac{\partial}{\partial w} J(w, b) = 0 \\ \frac{\partial}{\partial b} J(w, b) = 0 \end{cases}$$

即可求得最优参数

$$w^* = \frac{\frac{1}{n} \sum_{i=1}^n x_i y_i - \bar{x} \bar{y}}{\frac{1}{n} \sum_{i=1}^n x_i^2 - \bar{x}^2}$$
$$b^* = \bar{y} - \bar{x} w^*$$

线性回归-梯度下降法模型求解



梯度下降的基本思想是通过迭代过程逐步调整模型参数，使得目标函数的值逐渐减小，具体来说，算法从一个初始点开始，不断沿着目标函数梯度的反方向移动一小步最终达到一个局部或全局最优解。

梯度方向是增长速率最大的方向，其计算公式如下所示：

$$\nabla f(x, y, z) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right) = \frac{\partial f}{\partial x} \mathbf{i} + \frac{\partial f}{\partial y} \mathbf{j} + \frac{\partial f}{\partial z} \mathbf{k}$$

注：i、j、k分别为x、y、z轴的单位向量

例：对于 $J(w, b) = \frac{1}{2n} \sum_{i=1}^n (h(x^{(i)}) - y^{(i)})^2$ 其梯度分别为：

$$\frac{\partial J}{\partial w} = -\frac{1}{n} \sum_{i=1}^n x_i (y_i - \hat{y}_i)$$

$$\frac{\partial J}{\partial b} = -\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

线性回归-梯度下降法模型求解

13

通过如下两个更新公式，即可通过不断迭代找到最优参数。

学习率

梯度

$$w \leftarrow w - \alpha \frac{\partial L}{\partial w}$$
$$b \leftarrow b - \alpha \frac{\partial L}{\partial b}$$

$$w := w - \alpha \frac{1}{n} x_i \sum_{i=1}^n (\hat{y}_i - y_i)$$
$$b := b - \alpha \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

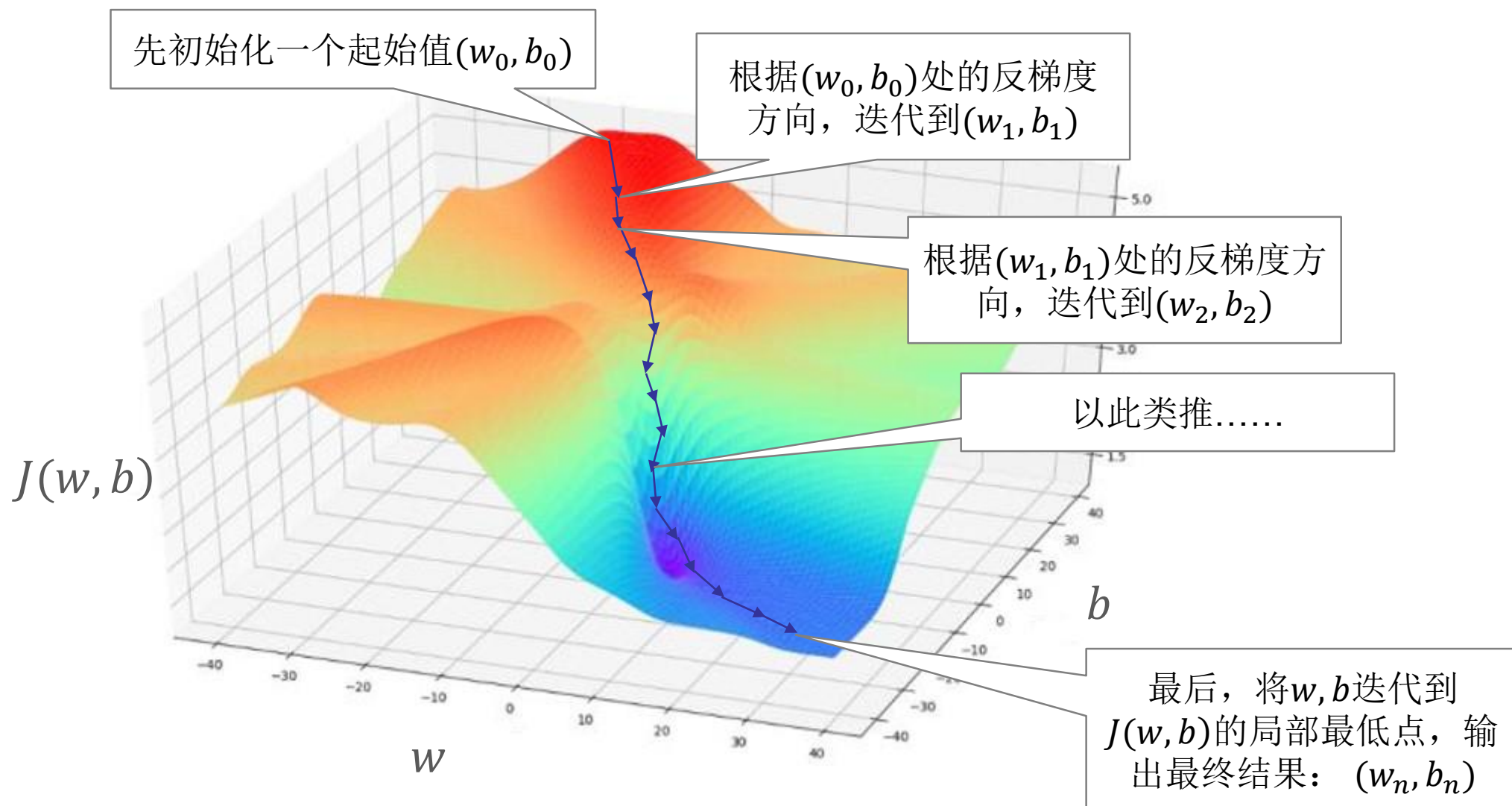
其中， α 为学习率，它决定了模型在训练过程中更新权重参数的速度。

线性回归-梯度下降法模型求解



北京邮电大学
Beijing University of Posts and Telecommunications

14



线性回归-多变量模型求解

15

建筑面积	总层数	楼层	实用面积	房价
143.7	31	10	105	36200
162.2	31	8	118	37000
199.5	10	10	170	42500
96.5	31	13	74	31200
.....				

$x^{(i)}$ 是特征矩阵中的第 i 行，是一个向量。

上图的：

$$x^{(2)} = \begin{bmatrix} 162.2 \\ 31 \\ 8 \\ 118 \end{bmatrix} \quad y^{(2)} = 37000$$

$x_j^{(i)}$ 代表特征矩阵中第 i 行的第 j 个特征

在实际问题中，状态量的变化往往由多种因素引起，如房价除受到房屋面积影响外，还受到楼层、建筑面积等属性的影响，为了表示目标变量与多个属性的线性关系，需要使用多维线性模型。

$$h(x) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

其向量形式为

$$h(x) = w^T x + b$$

线性回归-特征方程求解



要找到一组 $w(w_0 = b, w_1, w_2, \dots, w_n)$, 使得 $J(w) = \frac{1}{2n} \sum_{i=1}^n (h(x^{(i)}) - y^{(i)})^2$

(残差平方和) 最小, 即最小化: $\frac{\partial J(w)}{\partial w}$

将向量表达形式转为矩阵表达形式, 则有 $J(w) = \frac{1}{2} (Xw - Y)^2$, 其中 X 为 m 行 $n + 1$ 列的矩阵 (m 为样本个数, n 为特征个数) , w 为 $n + 1$ 行 1 列的矩阵(包含了 w_0) , Y 为 m 行 1 列的矩阵, 则 $J(w) = \frac{1}{2} (Xw - Y)^2 = \frac{1}{2} (Xw - Y)^T (Xw - Y)$

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & x_3^{(1)} & \dots & x_n^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & x_3^{(2)} & \dots & x_n^{(2)} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_1^{(m)} & x_2^{(m)} & x_3^{(m)} & \dots & x_n^{(m)} \end{bmatrix} \quad Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \dots \\ y^{(m)} \end{bmatrix}$$

需要用到向量平方的性质:

$$\sum_i z_i^2 = z^T z$$

线性回归-特征方程求解



为最小化，接下来对 $J(w)$ 求偏导，

$$\frac{\partial J(w)}{\partial w} = \frac{1}{2} \frac{\partial}{\partial w} (Xw - Y)^T (Xw - Y) = \frac{1}{2} \frac{\partial}{\partial w} (w^T X^T X w - Y^T X w - w^T X^T Y + Y^T Y)$$

由于中间两项互为转置:

$$\frac{\partial J(w)}{\partial w} = \frac{1}{2} \frac{\partial}{\partial w} (w^T X^T X w - 2w^T X^T Y + Y^T Y) = \frac{1}{2} (2X^T X w - 2X^T Y + 0)$$

$$= X^T X w - X^T Y$$

$$\text{令 } \frac{\partial J(w)}{\partial w} = 0,$$

$$\text{则有 } w = (X^T X)^{-1} X^T Y$$

需要用到以下几个矩阵的求导法则:

$$\frac{\partial X^T X}{\partial X} = 2X \quad \frac{\partial AX}{\partial X} = A^T$$

$$\frac{\partial X^T AX}{\partial X} = (A + A^T)X, \text{ 若 } A \text{ 为对称阵, } \frac{\partial X^T AX}{\partial X} = 2AX$$

我们可以使用处理一元线性回归函数的梯度下降法来处理多元线性回归函数，其后续的所有步骤（求解、预测等）并没有任何变化与不同，仅仅只是有了更多的特征属性。

由于使用特征方程求解 $(X^T X)^{-1}$ 的运算速度慢，时间复杂度大约是 $O(n^3)$ ，大多数情况下我们会选择梯度下降法。

$$w = (X^T X)^{-1} X^T Y$$

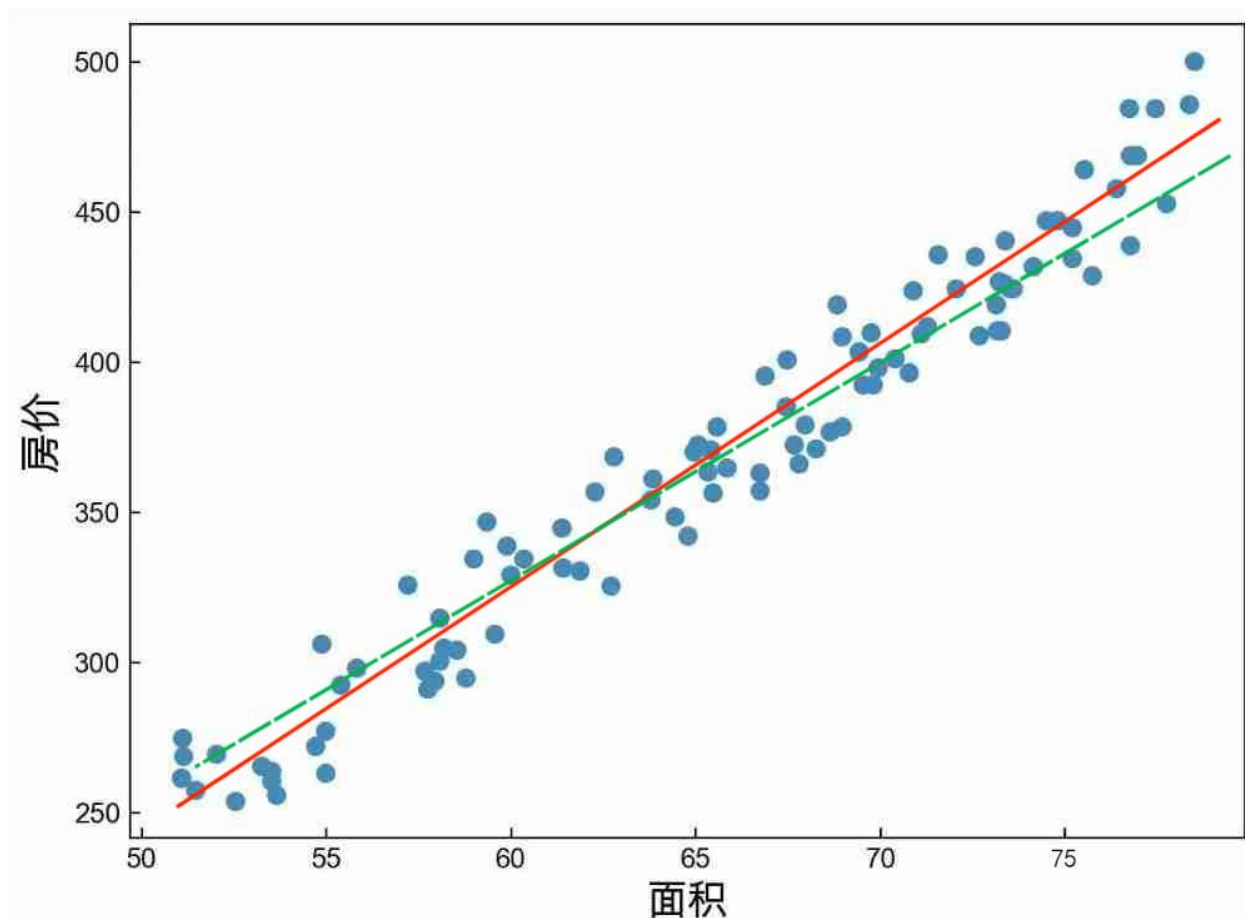
01 线性回归的概念

02 线性回归模型的建立

03 线性回归模型的求解

04 线性回归评价指标

线性回归-性能评估



以最开始的房价预测为例，
现在假设你求解得到了如图
所示的两个模型 $h_1(x)$ 与 $h_2(x)$
，那么应该选哪一个呢？

亦或是在不能可视化的情况
下，应该来如何评估模型的
好与坏呢？

平均绝对误差 (MAE)

MAE用来衡量预测值与真实值之间的平均绝对误差，定义如下：

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

其中MAE越小表示模型越好

平均绝对百分比误差 (MAPE)

MAPE和MAE类似，只是在MAE的基础上做了标准化处理，其定义如下：

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

其中MAPE越小表示模型越好。

均方误差 (MSE)

MSE用来衡量预测值与真实值之间的误差平方，定义如下：

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

其中MSE越小表示模型越好

R²评价指标

决定系数R²是线性回归模型中sklearn默认采用的评价指标，其定义如下：

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

其中，R²越大表示模型越好，表示真实值的平均值。

广义线性回归

实际应用中，大多数问题无法用线性模型准确地表示，将非线性的问题进行线性映射，就能够用线性回归的方法进行求解。

$$y = wx + b, \quad \ln(y) = wx + b$$

从等式的形式来看，对数线性回归与线性回归区别仅仅在于等式左部，形式依旧是线性回归，但实质上是完成了输入空间 x 到输出空间 y 的非线性映射。这里的对数函数 $\ln(\cdot)$ ，将线性回归模型和真实观测联系起来。

条件：映射的函数单调可微。

Linear Regression

<https://mlu-explain.github.io/linear-regression/>

Click the buttons to run 1, 10, or 100 steps of gradient descent, and see the linear regression model update live. The error at each iteration of gradient descent (or manual coefficient update) is shown in the bottom chart. With each weight update, we recalculate the error, so you can see how gradient descent improves our model iteratively.

New Data

1 Step

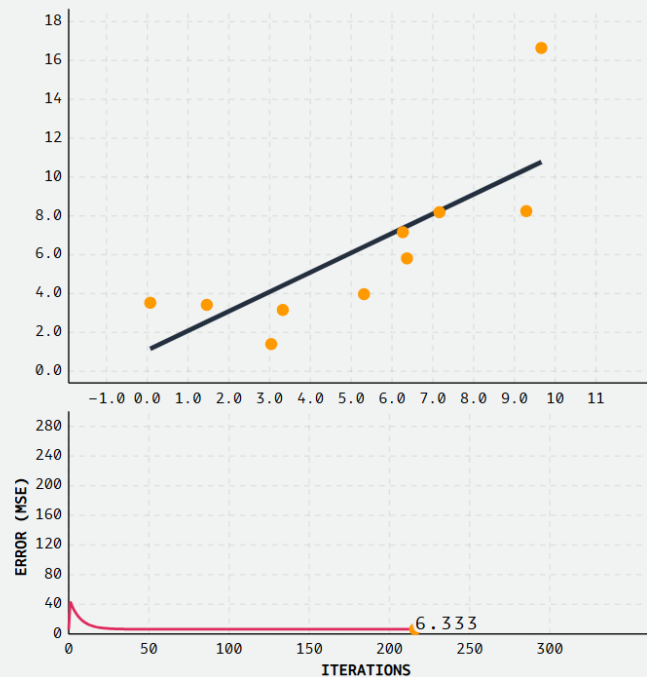
25 Steps

100 Steps

Bias ($\hat{\beta}_0$): 1.079

Weight ($\hat{\beta}_1$): 1.003

Our model: $y = 1.003x + 1.079 + c$





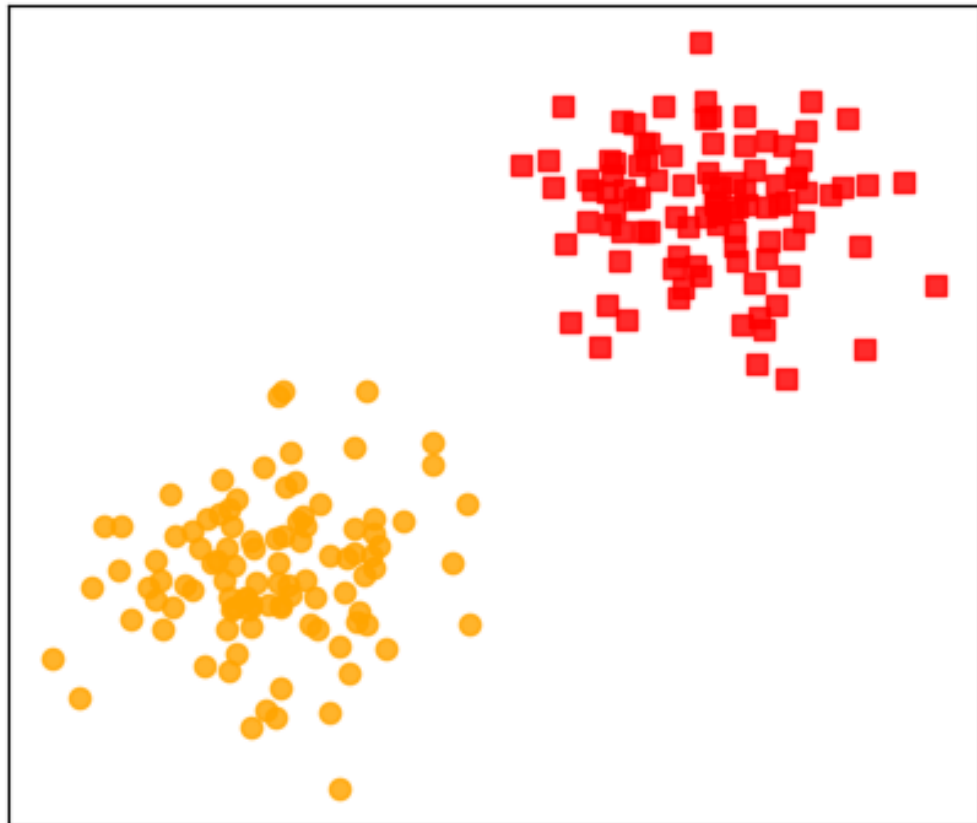
机器学习-逻辑回归

01 分类问题

02 逻辑回归模型的建立

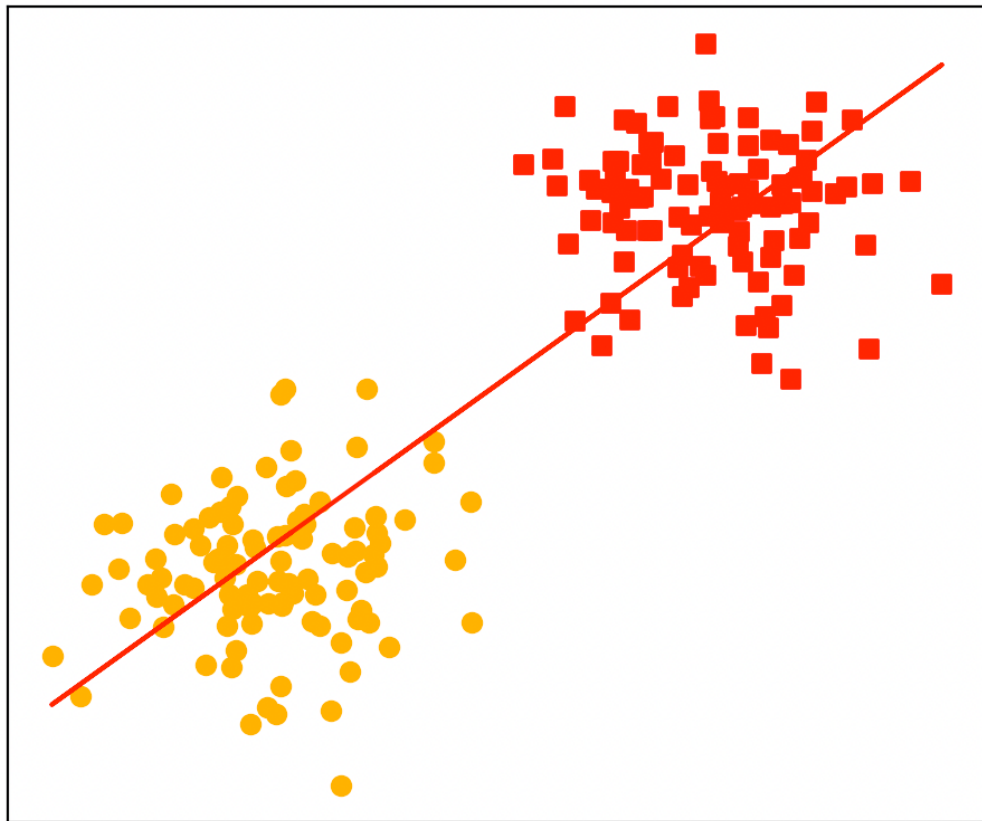
03 逻辑回归模型的求解

04 逻辑回归评价指标



如左图所示，现在有两堆样本点，需要建立一个模型来对新输入的样本进行预测，判断其应该属于那个类别，即**二分类问题（Binary Classification）**。对于这样一个问题我们可不可以通过前面的线性回归来解决呢？

对于这个问题用线性回归来解决肯定是不行的，因为两者本就属于不同类型的问题。退一步讲，即使是用线性回归来建模得到也是一条向右倾斜的直线，而我们这里需要的却是一条向左倾斜的直线。



同时，回归模型的预测值都是位于预测曲线附近的样本点，而无法做到区分直线两边的东西。

那可不可以在此基础上做一点改进以实现分类的目的呢？

01 分类问题

02 逻辑回归模型的建立

03 逻辑回归模型的求解

04 逻辑回归评价指标

既然是解决二分类问题，那么完全可以通过建立一个模型用来预测每个样本点属于其中一个类别的概率 p ，如果 $p > 0.5$ 那我们就可以认为该样本点属于某个类别，这样就能解决上述的二分类问题。

怎么建立这样一个模型？

在线性回归中，通过建模 $h(x) = wx + b$ 来对新样本进行预测，其输出可为任意实数。但此处既然是要得到一个样本所属类别的概率，那最直接的办法就是通过一个函数 $g(z)$ ，将 $h(x)$ 映射至 $[0,1]$ 的范围。由此，便得到了逻辑回归中的预测模型：

$$\hat{y} = h(x) = g(wx + b)$$

前面介绍了可以通过一个函数 $g(z)$ 将特征的线性组合 $z = wx + b$ 映射到区间 $[0,1]$ 中去, 那么这个 $g(z)$ 长什么呢?

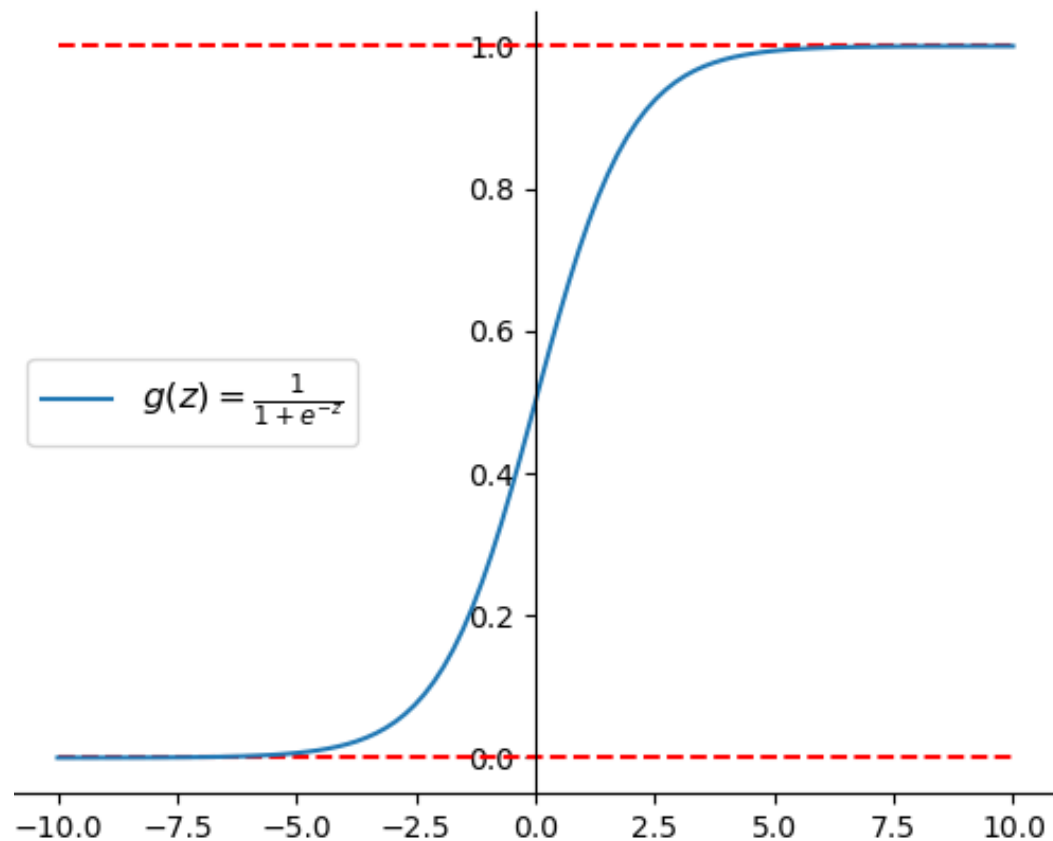
Sigmoid 函数

$g(z)$ 代表一个常用的逻辑函数 (logistic function) 为S形函数 (Sigmoid function)

$$\text{且: } g(z) = \frac{1}{1+e^{-z}}$$

当 $g(z)$ 大于等于0.5时, 预测 $y=1$

当 $g(z)$ 小于0.5时, 预测 $y=0$



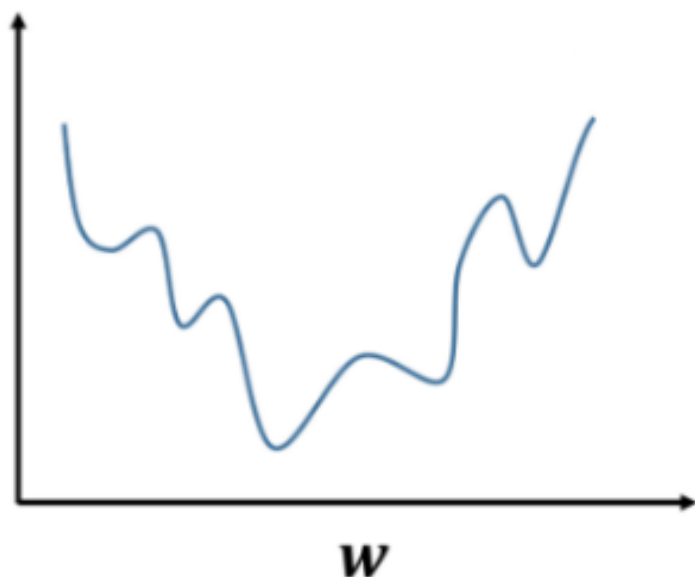
01 分类问题

02 逻辑回归模型的建立

03 逻辑回归模型的求解

04 逻辑回归评价指标

能否和线性回归一样 选择均方误差作为代价函数？



$$Cost(w) = MSE = \frac{1}{n} \sum_{i=1}^n (h(x_i) - y_i)^2 = \frac{1}{n} \sum_{i=1}^n \left(y_i - \frac{1}{1 + e^{-(wx+b)}} \right)^2$$

由图可知，直接通过MSE构造的代价函数非凸，若学习率设置不合理，梯度下降法仅能找到局部最小值，很难找到全局最小值。

逻辑回归-损失函数

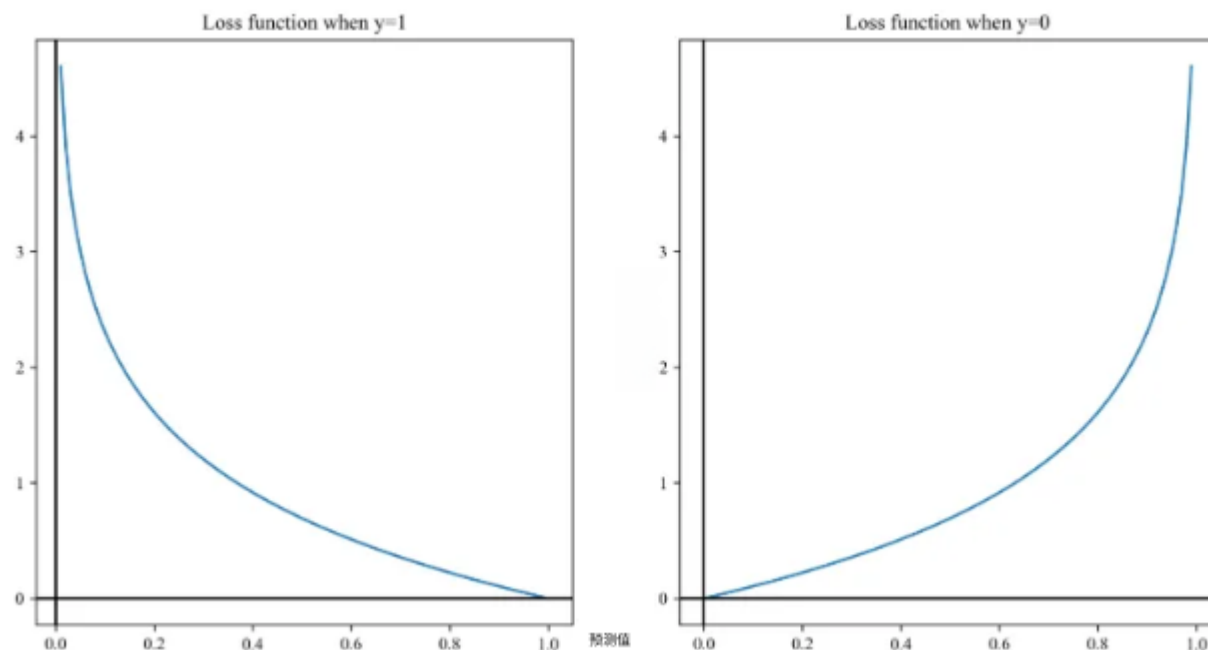
35

损失函数

$$L(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

$\hat{y}$ 表示预测值 $h(x)$

y 表示真实值



左图中横坐标为预测值 $\hat{y}$
纵坐标为损失函数 L

当 $y=1$ 时, 可以看到预测值越接近 1, 即分类越接近正确, 损失越小; 预测值越接近 0, 即分类越错误, 损失越大。
当 $y=0$ 时, 可以看到预测值越接近 0, 即分类越接近正确, 损失越小; 预测值越接近 1, 即分类越错误, 损失越大。

对于在上面给出的损失函数, 我们是如何得到的呢?

设

$$P(y = 1 | x; w, b) = h(x) = \frac{1}{1 + e^{-(wx+b)}}$$

$$P(y = 0 | x; w, b) = 1 - h(x) = \frac{e^{-(wx+b)}}{1 + e^{-(wx+b)}} = \frac{1}{1 + e^{wx+b}}$$

$$h(x) = g(z) = g(wx + b)$$

$P(y = 1 | x; W, b) = h(x)$ 的含义为当给定参数 w, b 时, 样本 x 属于 $y = 1$ 这个类别的概率为 $h(x)$

- 进一步可以将上面的3个表达式改为

$$p(y | x; w, b) = (h(x))^y (1 - h(x))^{1-y}$$

将上式取对数, 即可得到逻辑回归的损失函数

$$L(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

那么什么样的参数 w, b 能够使得已知的 $y^{(1)}, y^{(2)}, \dots, y^{(m)}$, 这样一个结果(分布) 最容易出现呢? 为了能够使得 $y^{(1)}, y^{(2)}, \dots, y^{(m)}$ 这样一个结果最容易出现, 根据**极大似然估计**的思想, 我们应当最大化如下似然函数

$$L(w, b) = \prod_{i=1}^n p(y^{(i)} | x^{(i)}; w, b) = \prod_{i=1}^n (h(x^{(i)}))^{y^{(i)}} (1 - h(x^{(i)}))^{1-y^{(i)}}$$

两边同时取自然对数有

$$\ell(w, b) = \log L(w, b) = \sum_{i=1}^n [y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log (1 - h(x^{(i)}))]$$

对上式再取负平均, 即可得到逻辑回归的目标函数:

$$J(w, b) = -\frac{1}{n} \sum_{i=1}^n [y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log (1 - h(x^{(i)}))]$$

在逻辑回归模型中，我们的目标是求解合适的参数，使得模型在训练样本上能够获得最小的误差。具体而言，就是要找到最优的权重参数 w 和偏置参数 b ，通过不断调整它们的值，让模型的预测结果尽可能地接近训练样本的真实标签，从而实现最小化误差的目的。

$$J(w, b) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)})) \right]$$
$$h(x^{(i)}) = g(wx^{(i)} + b)$$

其中， m 表示样本总数， $x^{(i)}$ 表示第 i 个样本， $y^{(i)}$ 表示第 i 个样本的真实标签， $h(x^{(i)})$ 表示第 i 个样本为正类的预测概率。

目标函数推导——求解梯度



通过梯度下降算法可以最小化某个目标函数。当目标函数取得（或接近）最小值时，便得到了目标函数中对应的未知参数。所以就要求解得到目标函数关于各个参数的梯度。

$$J(w, b) = -\frac{1}{n} \sum_{i=1}^n [y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))]$$

目标函数 $J(w, b)$ 关于参数 w 的梯度为：

$$\begin{aligned} \frac{\partial J}{\partial w} &= -\frac{\partial}{\partial w} \frac{1}{n} \sum_{i=1}^n [y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))] \\ &= -\frac{1}{n} \sum_{i=1}^n \left[y^{(i)} \frac{h'(x^{(i)})}{h(x^{(i)})} + (1 - y^{(i)}) \frac{-h'(x^{(i)})}{1 - h(x^{(i)})} \right] \\ &= -\frac{1}{n} \sum_{i=1}^n \left[y^{(i)} \frac{g(z^{(i)})(1 - g(z^{(i)}))}{g(z^{(i)})} x_j^{(i)} - (1 - y^{(i)}) \frac{g(z^{(i)})(1 - g(z^{(i)}))}{1 - g(z^{(i)})} x_j^{(i)} \right] \\ &= -\frac{1}{n} \sum_{i=1}^n [y^{(i)}(1 - g(z^{(i)})) - (1 - y^{(i)})g(z^{(i)})] x_j^{(i)} \\ &= -\frac{1}{n} \sum_{i=1}^n [y^{(i)} - h(x^{(i)})] x_j^{(i)} \end{aligned}$$

目标函数推导——求解梯度



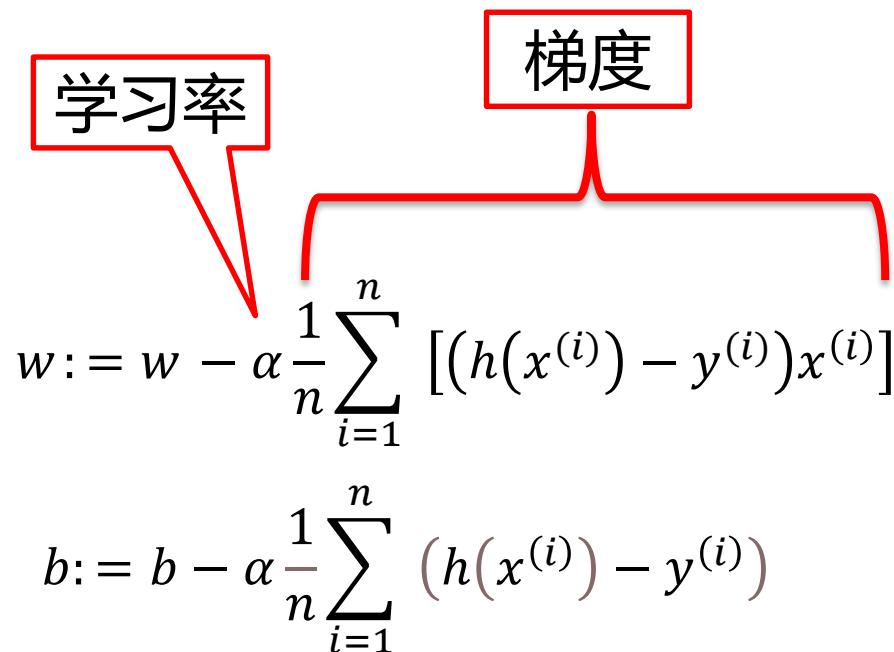
$$J(w, b) = -\frac{1}{n} \sum_{i=1}^n [y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))]$$

目标函数 $J(w, b)$ 关于参数 b 的梯度为：

$$\begin{aligned} \frac{\partial J}{\partial b} &= -\frac{\partial}{\partial b} \frac{1}{n} \sum_{i=1}^n [y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))] \\ &= -\frac{1}{n} \sum_{i=1}^n \left[y^{(i)} \frac{h'(x^{(i)})}{h(x^{(i)})} + (1 - y^{(i)}) \frac{-h'(x^{(i)})}{1 - h(x^{(i)})} \right] \\ &= -\frac{1}{n} \sum_{i=1}^n \left[y^{(i)} \frac{g(z^{(i)})(1 - g(z^{(i)}))}{g(z^{(i)})} - (1 - y^{(i)}) \frac{g(z^{(i)})(1 - g(z^{(i)}))}{1 - g(z^{(i)})} \right] \\ &= -\frac{1}{n} \sum_{i=1}^n [y^{(i)}(1 - g(z^{(i)})) - (1 - y^{(i)})g(z^{(i)})] \\ &= -\frac{1}{n} \sum_{i=1}^n [y^{(i)} - h(x^{(i)})] \end{aligned}$$

目标函数推导——参数更新

通过如下两个更新公式，即可通过不断迭代找到最优参数。



The diagram shows two update formulas with red boxes and arrows highlighting specific parts. The first formula is $w := w - \alpha \frac{1}{n} \sum_{i=1}^n [(h(x^{(i)}) - y^{(i)})x^{(i)}]$. A red box labeled '学习率' (Learning Rate) points to the α term. Another red box labeled '梯度' (Gradient) points to the entire term $\frac{1}{n} \sum_{i=1}^n [(h(x^{(i)}) - y^{(i)})x^{(i)}]$. The second formula is $b := b - \alpha \frac{1}{n} \sum_{i=1}^n (h(x^{(i)}) - y^{(i)})$. A red box labeled '梯度' points to the entire term $\frac{1}{n} \sum_{i=1}^n (h(x^{(i)}) - y^{(i)})$.

$$w := w - \alpha \frac{1}{n} \sum_{i=1}^n [(h(x^{(i)}) - y^{(i)})x^{(i)}]$$
$$b := b - \alpha \frac{1}{n} \sum_{i=1}^n (h(x^{(i)}) - y^{(i)})$$

其中， α 为学习率，它决定了模型在训练过程中更新权重参数的速度。

1. 优点

- (1) 逻辑回归模型**形式简单，可解释性好**，从特征的权重可以看到不同的特征对最后结果的影响。
- (2) 训练时便于**并行化**，在预测时只需要对特征进行线性加权，所以性能比较好，往往适合处理海量ID类特征，用ID类特征有一个很重要的好处，就是防止信息损失(相对于范化的 CTR 特征)，对于头部资源会有更细致的描述。
- (3) **资源占用小**，尤其是内存。在实际的工程应用中只需要存储权重比较大的特征及特征对应的权重。
- (4) **方便输出结果调整**。逻辑回归可以很方便的得到最后的分类结果，因为输出的是每个样本的概率分数，我们可以很容易的对这些概率分数进行划分阈值(大于某个阈值的是一类，小于某个阈值的是一类)。

2. 缺点

(1) **表达能力不强**，无法进行特征交叉，特征筛选等一系列“高级”（这些工作都得依赖有经验的人来完成，否则会走一些弯路），因此可能造成信息的损失。这就需要开发者有非常丰富的领域经验，才能不走弯路。这样的模型迁移起来比较困难，换一个领域又需要重新进行大量的特征工程。

(2) **准确率并不是很高**。因为这毕竟是一个线性模型加了个Sigmoid函数，形式非常的简单(非常类似线性模型)，很难去拟合数据的真实分布。

(3) **处理非线性数据较麻烦**。逻辑回归在不引入其他方法的情况下，只能处理线性可分的数据，如果想处理非线性，首先对连续特征的处理需要先进行离散化(离散化的目的是为了引入非线性)，人工分箱的方式会引入多种问题。

01 分类问题

02 Sigmoid函数

03 逻辑回归求解

04 逻辑回归评价指标

如同回归模型一样，对于任何分类模型来讲同样需要通过一些评价指标来衡量模型的优与劣。在分类任务中，常见的**评价指标 (Metric)** 有：**准确率 (Accuracy)**、**精确率 (Precision)**、**召回率 (Recall)** 与 **F 值 (F_{score})**，其中应用最为广泛的的就是准确率，接着是召回率。

为了能够更加容易的理解与运用这4种评价指标，下面我们将会详细地对这4种指标进行介绍。

以猫狗图片识别场景为例，假设一个猫狗分类器对100张图片进行分类，分类结果显示38张图片是猫，62张图片是狗。经与真实标签对比发现，38张猫的图片中有20张是分类正确的，62张狗的图片中有57张是分类正确的。

根据上述描述可得到如下所示的矩阵，称为**混淆矩阵 (Confusion Matrix)**

真实 \ 预测	猫	狗
	猫	狗
猫	20	5
狗	18	57

真实 \ 预测	P	N
	P	N
P	TP	FN
N	FP	TN

如何来读这个混淆矩阵呢？读的时候**首先横向看，然后纵向看**。例如读TP的时候，首先横向表示真实的正样本，其次纵向表示预测的正样本，因此TP表示的就是将正样本预测为正样本的个数，即预测正确。同理有：

真实 \ 预测	P	N
	P	N
P	TP	FN
N	FP	TN

True Positive (TP): 将正样本预测为正样本，即预测正确；
False Negative (FN): 将正样本预测为负样本，即预测错误；
False Positive (FP): 将负样本预测为正样本，即预测错误；
True Negative (TN): 将负样本预测为负样本，即预测正确。

在定义完上述4个类别的分类情况后就能够定义出各种场景下的计算指标

真实 \ 预测	P	N
	P	N
P	TP	FN
N	FP	TN

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F_{\text{score}} = (1 + \beta^2) \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}$$

准确率 (**Accuracy**) 最容易理解, 即所有预测正确的样本占总样本的比例

精确率 (**Precision**) 是计算被预测为正类的样本中, 实际为正样本的比例

召回率 (**Recall**) 是计算所有实际为正类的样本中, 被正确预测为正类的比例

而F值 (**F_{score}**) 则是精确率与召回率的调和平均

值得注意的是，通常在绝大多数任务中并不会明确哪一类别是正样本，哪一类别又是负样本，所以对于每个类别来说都可以计算其各项指标（但是准确率只有一个）。

真实 \ 预测	猫	狗
	猫	狗
猫	20	5
狗	18	57

真实 \ 预测	P	N
	P	N
P	TP	FN
N	FP	TN

$$\text{Accuracy} = \frac{20+57}{20+18+5+57} = 0.77$$

□ 对于类别猫来说

$$\text{Precision} = \frac{20}{20+18} = 0.53$$

$$\text{Recall} = \frac{20}{20+5} = 0.8$$

$$F_{\text{score}} = \frac{2 \times 0.53 \times 0.8}{0.53 + 0.8} = 0.63$$

真实 \ 预测	猫	狗
	猫	狗
猫	20	5
狗	18	57

真实 \ 预测	P	N
	P	N
P	TP	FN
N	FP	TN

□ 对于类别狗来说

$$\text{Precision} = \frac{57}{57+5} = 0.92$$

$$\text{Recall} = \frac{57}{57+18} = 0.76$$

$$F_{\text{score}} = \frac{2 \times 0.92 \times 0.76}{0.92 + 0.76} = 0.83$$

Logistic Regression

<https://mlu-explain.github.io/logistic-regression/>

Let's see how gradient descent works for our logistic regression model. We'll use the algorithm to identify which values for bias ($\hat{\beta}_0$) and weight ($\hat{\beta}_1$) we should select. Click the buttons to run 1, 5, 10, or 25 steps of gradient descent, and see the curve update live. The error for each iteration is shown in the bottom error chart.

1 Step

5 Steps

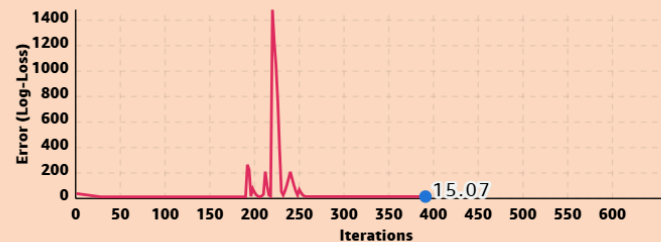
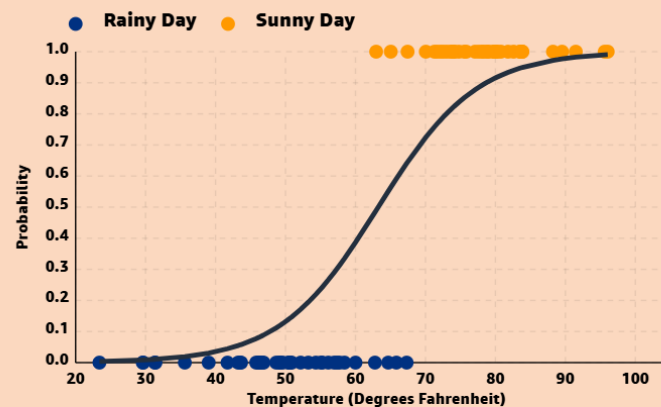
10 Steps

25 Steps

Weight: 0.1423

Bias: -9.0001

Our Model: $P(y = 1|x) = \frac{1}{1 + e^{-(-9.0001 + 0.1423x)}}$



线性回归：线性回归是一种常用于回归问题的算法。其目标是通过建立自变量（输入特征）与因变量（目标值）之间的线性关系来进行预测。线性回归的目的是预测一个连续的数值输出。例如，预测房价、股票价格、温度等。

逻辑回归：逻辑回归尽管名字中带有“回归”二字，但它是一种分类算法，用于二分类问题。逻辑回归的目标是预测输入数据属于某个类别的概率，最终输出是一个概率值，用于将样本分类到两个类别之一。逻辑回归最常见的应用场景是处理“是/否”类型的问题，如疾病诊断、客户流失预测、垃圾邮件识别等。

线性回归：模型表示为

$$h(x)wx + b \quad h(x) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

输出范围为实数 $(-\infty, +\infty)$ 。

逻辑回归：模型表示为：

$$\hat{y} = h(x) = g(wx + b)$$

输出范围为概率 $(0, 1)$ 。



```
import matplotlib.pyplot as plt  
#引入画图使用的工具包matplotlib.pyplot
```

```
x = [325.68, 331.15, 338.69, 345.90, 354.19, 360.88, 369.48, 379.67]  
y = [-0.108, -0.082, 0.015, 0.080, 0.149, 0.240, 0.370, 0.420]  
fig = plt.figure()  
plt.xlabel("X")  
plt.ylabel("Y")  
plt.scatter(x,y,c='r')  
plt.show()
```

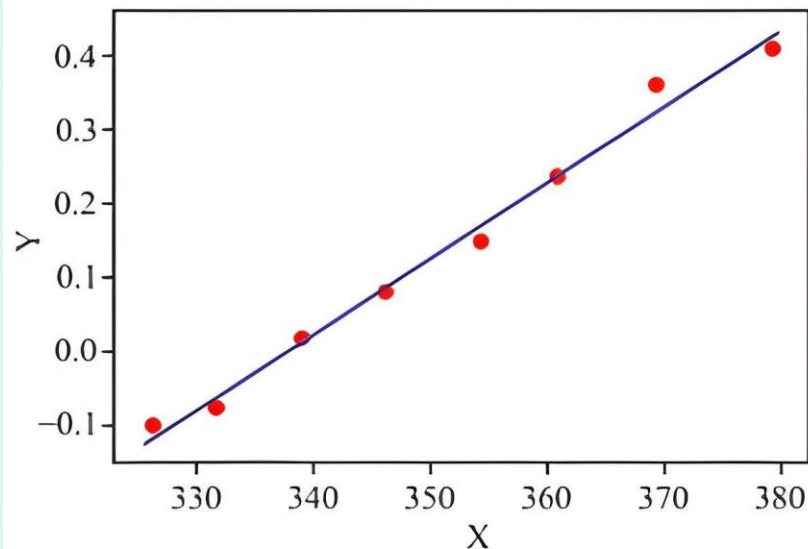




```

import matplotlib.pyplot as plt
import numpy as np
#引入使用的工具包matplotlib和numpy
x = [325.68, 331.15, 338.69, 345.90, 354.19, 360.88, 369.48, 379.67]
y = [-0.108, -0.082, 0.015, 0.080, 0.149, 0.240, 0.370, 0.420]
x_avarage = sum(x)/len(x)
y_avarage = sum(y)/len(y)                                #计算两个平均数
m = 0
n = 0                                                    #两个临时变量用于后续计算参数a和b
for i in range(len(x)):
    m += x[i]*y[i]
    n += x[i]*x[i]
a = ( m-len(x)*x_avarage*y_avarage) / ( n-len(x)*x_avarage*x_avarage)
b = y_avarage - a*x_avarage                              #计算参数a和b
x_predict = np.linspace(325,380,1000)
y_predict = a*x_predict + b                             #按照上面计算出的参数构造直线
fig = plt.figure()
plt.xlabel("X")
plt.ylabel("Y")
plt.scatter(x,y,c='r')
plt.plot(x_predict,y_predict,c='b')
plt.show()

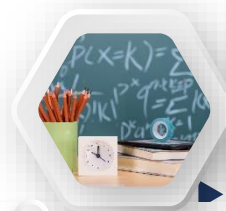
```



二氧化碳浓度和温度散点图及拟合直线图

$$w^* = \frac{\frac{1}{n} \sum_{i=1}^n x_i y_i - \bar{x} \bar{y}}{\frac{1}{n} \sum_{i=1}^n x_i^2 - \bar{x}^2}$$

$$b^* = \bar{y} - \bar{x} w^*$$



北京邮电大学

谢谢!